

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра информационных технологий и автоматизированных систем

ОТЧЕТ
о лабораторной работе 4
Семестр: 4

На тему: «Автоматы»

Выполнил студент ИВТ-24-26:
Шишкин М.Г

(дата, подпись)

Проверил:

Рустамханова Г.И

(дата, подпись)

Пермь 2026

Содержание

1 Постановка задачи	3
2 Краткое описание решения	4
3 Код	5
4 Тестирование программы	10
5 Вывод	12
6 Github	13

1 Постановка задачи

Формальный язык задан в алфавите $\{a, b, c, d, e\}$ и содержит слова любой длины, содержащие хотя бы одно вхождение трех одинаковых символов подряд, начинающиеся и заканчивающиеся на гласную (а или е). Требуется синтезировать автомат, распознающий заданный язык, и написать программу-анализатор, которая определяет, принадлежит ли слово заданному языку

2 Краткое описание решения

Автомат реализован как детерминированный конечный автомат (ДКА) без использования счетчиков. Формальное описание автомата: $M = (Q, \Sigma, \delta, q_0, F)$, где $Q = \{S, A1, B1, C1, D1, E1, A2, B2, C2, D2, E2, Fa, Fb, Fc, Fd, Fe, R\}$ — множество из 17 состояний, $\Sigma = \{a, b, c, d, e\}$ — входной алфавит, δ — функция переходов, заданная матрицей 17×5 , $q_0 = S$ — начальное состояние, $F = \{Fa, Fe\}$ — множество допускающих состояний. Регулярное выражение языка: $L = (a+e)(a+b+c+d+e)(aaa+bbb+ccc+ddd+eee)(a+b+c+d+e)(a+e)$. Автомат читает слово посимвольно, меняя состояние согласно матрице переходов. Состояния кодируют последний прочитанный символ, количество одинаковых символов подряд, наличие трех одинаковых, а также информацию о первом и последнем символе. Допуск происходит только в состояниях Fa и Fe , что соответствует наличию трех одинаковых символов и окончанию слова на гласную.

3 Код

```
1  #include <iostream>
2  #include <string>
3  #include <iomanip>
4
5  using namespace std;
6
7  // Состояния автомата (18 состояний)
8  enum State {
9      S, // начальное
10     A1, B1, C1, D1, E1, // последний символ (один)
11     A2, B2, C2, D2, E2, // последние два одинаковых
12     Fa, Fb, Fc, Fd, Fe, // найдено 3 одинаковых
13     R, // reject
14     NUM_STATES // ВСЕГО СОСТОЯНИЙ (должно быть последним!)
15 };
16
17 // Явно задаём количество состояний
18 const int STATE_COUNT = 18;
19
20 const char* stateNames[STATE_COUNT] = {
21     "S", "A1", "B1", "C1", "D1", "E1",
22     "A2", "B2", "C2", "D2", "E2",
23     "Fa", "Fb", "Fc", "Fd", "Fe",
24     "R"
25 };
26
27 // Преобразование символа в индекс (0-4)
28 int toIdx(char c) {
29     switch (c) {
30         case 'a': return 0;
31         case 'b': return 1;
32         case 'c': return 2;
33         case 'd': return 3;
34         case 'e': return 4;
35         default: return -1;
36     }
37 }
38
39 // МАТРИЦА ПЕРЕХОДОВ
40 State delta[STATE_COUNT][5] = {
41     /* 0 S */ {A1, R, R, R, E1},
42     /* 1 A1 */ {A2, B1, C1, D1, E1},
43     /* 2 B1 */ {A1, B2, C1, D1, E1},
44     /* 3 C1 */ {A1, B1, C2, D1, E1},
45     /* 4 D1 */ {A1, B1, C1, D2, E1},
46     /* 5 E1 */ {A1, B1, C1, D1, E2},
47     /* 6 A2 */ {Fa, B1, C1, D1, E1},
48     /* 7 B2 */ {A1, Fb, C1, D1, E1},
49     /* 8 C2 */ {A1, B1, Fc, D1, E1},
50     /* 9 D2 */ {A1, B1, C1, Fd, E1},
51     /*10 E2 */ {A1, B1, C1, D1, Fe},
52     /*11 Fa */ {Fa, Fb, Fc, Fd, Fe},
53     /*12 Fb */ {Fa, Fb, Fc, Fd, Fe},
54     /*13 Fc */ {Fa, Fb, Fc, Fd, Fe},
```

```

55     /*14 Fd */ {Fa, Fb, Fc, Fd, Fe},
56     /*15 Fe */ {Fa, Fb, Fc, Fd, Fe},
57     /*16 R  */ {R, R, R, R, R}
58 };
59
60 // Проверка индекса состояния
61 bool isValidState(State s) {
62     return (s >= 0 && s < STATE_COUNT);
63 }
64
65 // Допускающие состояния (только Fa и Fe)
66 bool isAccepting(State s) {
67     return (s == Fa || s == Fe);
68 }
69
70 // =====
71 // ВЫВОД ТЕОРИИ
72 // =====
73
74 void printRegex() {
75     cout << "\n===== " << endl;
76     cout << "РЕГУЛЯРНОЕ ВЫРАЖЕНИЕ" << endl;
77     cout << "===== " << endl;
78     cout << "L = (a+e) (a+b+c+d+e)* (aaa+bbb+ccc+ddd+eee) (a+b+c+d+e)* (a+e)" << endl;
79     cout << "===== \n" << endl;
80 }
81
82 void printStates() {
83     cout << "\n===== " << endl;
84     cout << "ОПИСАНИЕ СОСТОЯНИЙ" << endl;
85     cout << "===== " << endl;
86     cout << "S - начальное состояние" << endl;
87     cout << "A1 - последний символ 'a'" << endl;
88     cout << "B1 - последний символ 'b'" << endl;
89     cout << "C1 - последний символ 'c'" << endl;
90     cout << "D1 - последний символ 'd'" << endl;
91     cout << "E1 - последний символ 'e'" << endl;
92     cout << "A2 - последние два символа 'aa'" << endl;
93     cout << "B2 - последние два символа 'bb'" << endl;
94     cout << "C2 - последние два символа 'cc'" << endl;
95     cout << "D2 - последние два символа 'dd'" << endl;
96     cout << "E2 - последние два символа 'ee'" << endl;
97     cout << "Fa - найдено 3 одинаковых, последний символ 'a' (ДОПУСК)" << endl;
98     cout << "Fb - найдено 3 одинаковых, последний символ 'b' (НЕ ДОПУСК)" << endl;
99     cout << "Fc - найдено 3 одинаковых, последний символ 'c' (НЕ ДОПУСК)" << endl;
100    cout << "Fd - найдено 3 одинаковых, последний символ 'd' (НЕ ДОПУСК)" << endl;
101    cout << "Fe - найдено 3 одинаковых, последний символ 'e' (ДОПУСК)" << endl;
102    cout << "R - состояние ошибки (отказ)" << endl;
103    cout << "===== \n" << endl;
104 }
105
106 void printTransitionTable() {
107     cout << "\n===== " << endl;
108     cout << "ТАБЛИЦА ПЕРЕХОДОВ (S: Q x Σ → Q)" << endl;

```

```

109 cout << "===== " << endl;
110 cout << "\n | a b c d e" << endl;
111 cout << "-----" << endl;
112
113 for (int i = 0; i < STATE_COUNT - 1; i++) { // -1 чтобы не выводить NUM_STATES
114     cout << stateNames[i] << " | ";
115     for (int j = 0; j < 5; j++) {
116         State nextState = delta[i][j];
117         if (isValidState(nextState)) {
118             cout << stateNames[nextState] << " ";
119         }
120         else {
121             cout << "? ";
122         }
123     }
124     cout << endl;
125 }
126 cout << "===== \n" << endl;
127 }
128
129 void printGraph() {
130     cout << "\n===== " << endl;
131     cout << "ГРАФ ПЕРЕХОДОВ" << endl;
132     cout << "===== " << endl;
133     cout << "Начальное состояние: S" << endl;
134     cout << "Допускающие состояния: Fa, Fe" << endl;
135     cout << endl;
136     cout << "Основные переходы:" << endl;
137     cout << " S -a-> A1 S -e-> E1 S -b,c,d-> R" << endl;
138     cout << " A1 -a-> A2 A1 -b-> B1 A1 -c-> C1 A1 -d-> D1 A1 -e-> E1" << endl;
139     cout << " B1 -a-> A1 B1 -b-> B2 B1 -c-> C1 B1 -d-> D1 B1 -e-> E1" << endl;
140     cout << " C1 -a-> A1 C1 -b-> B1 C1 -c-> C2 C1 -d-> D1 C1 -e-> E1" << endl;
141     cout << " D1 -a-> A1 D1 -b-> B1 D1 -c-> C1 D1 -d-> D2 D1 -e-> E1" << endl;
142     cout << " E1 -a-> A1 E1 -b-> B1 E1 -c-> C1 E1 -d-> D1 E1 -e-> E2" << endl;
143     cout << endl;
144     cout << "Обнаружение трёх одинаковых:" << endl;
145     cout << " A2 -a-> Fa B2 -b-> Fb C2 -c-> Fc D2 -d-> Fd E2 -e-> Fe" << endl;
146     cout << endl;
147     cout << "После обнаружения (запоминаем последний символ):" << endl;
148     cout << " Fa -a-> Fa Fa -b-> Fb Fa -c-> Fc Fa -d-> Fd Fa -e-> Fe" << endl;
149     cout << " Fb -a-> Fa Fb -b-> Fb Fb -c-> Fc Fb -d-> Fd Fb -e-> Fe" << endl;
150     cout << " Fc -a-> Fa Fc -b-> Fb Fc -c-> Fc Fc -d-> Fd Fc -e-> Fe" << endl;
151     cout << " Fd -a-> Fa Fd -b-> Fb Fd -c-> Fc Fd -d-> Fd Fd -e-> Fe" << endl;
152     cout << " Fe -a-> Fa Fe -b-> Fb Fe -c-> Fc Fe -d-> Fd Fe -e-> Fe" << endl;
153     cout << " R -любой-> R" << endl;
154     cout << "===== \n" << endl;
155 }
156
157 void printFormalDefinition() {
158     cout << "\n===== " << endl;
159     cout << "ФОРМАЛЬНОЕ ОПИСАНИЕ АВТОМАТА" << endl;
160     cout << "===== " << endl;
161     cout << "M = (Q, Σ, δ, q0, F)" << endl;
162     cout << endl;

```

```

163     cout << "Q = {S, A1, B1, C1, D1, E1, A2, B2, C2, D2, E2, Fa, Fb, Fc, Fd, Fe, R}" << endl;
164     cout << "Σ = {a, b, c, d, e}" << endl;
165     cout << "δ - функция переходов (см. таблицу выше)" << endl;
166     cout << "q0 = S" << endl;
167     cout << "F = {Fa, Fe} (допускающие состояния)" << endl;
168     cout << "=====\\n" << endl;
169 }
170
171 // =====
172 // РАСПОЗНАВАНИЕ
173 // =====
174
175 bool recognize(const string& w) {
176     if (w.empty()) {
177         cout << "Пустая строка не принадлежит языку" << endl;
178         return false;
179     }
180
181     State cur = S;
182
183     cout << "\\n--- ПРОЦЕСС РАБОТЫ АВТОМАТА ---" << endl;
184     cout << "Начальное состояние: " << stateNames[cur] << endl;
185
186     for (size_t i = 0; i < w.length(); i++) {
187         char c = w[i];
188         int idx = toIdx(c);
189
190         if (idx == -1) {
191             cout << "ОШИБКА: Недопустимый символ '" << c << "'" << endl;
192             return false;
193         }
194
195         // Проверим, что текущее состояние валидное
196         if (!isValidState(cur)) {
197             cout << "ОШИБКА: Невалидное состояние " << cur << endl;
198             return false;
199         }
200
201         State next = delta[cur][idx];
202
203         // Проверим, что следующее состояние валидное
204         if (!isValidState(next)) {
205             cout << "ОШИБКА: Невалидное следующее состояние " << next << endl;
206             return false;
207         }
208
209         cout << " Читаем '" << c << "': " << stateNames[cur] << " --" << c << "--> " << stateNames[next] << endl;
210         cur = next;
211
212         if (cur == R) {
213             cout << "Автомат перешёл в состояние REJECT (первый символ не гласная?)" << endl;
214             return false;
215         }
216     }

```



```

217     cout << "Конечное состояние: " << stateNames[cur] << endl;
218
219
220     if (isAccepting(cur)) {
221         cout << "✓ Состояние допускающее (есть 3 одинаковых и последний символ - гласная)" << endl;
222         return true;
223     }
224     else if (cur == Fb || cur == Fc || cur == Fd) {
225         cout << "✗ Найдено 3 одинаковых, НО последний символ - не гласная" << endl;
226         return false;
227     }
228     else if (cur == R) {
229         cout << "✗ Отказ" << endl;
230         return false;
231     }
232     else {
233         cout << "✗ Нет трёх одинаковых символов подряд" << endl;
234         return false;
235     }
236 }
237
238 // =====
239 // MAIN
240 // =====
241
242 int main() {
243     setlocale(LC_ALL, "Russian");
244
245     printRegex();
246     printFormalDefinition();
247     printStates();
248     printTransitionTable();
249     printGraph();
250
251     string input;
252     cout << "\n===== " << endl;
253     cout << "ПРОВЕРКА СЛОВ" << endl;
254     cout << "===== " << endl;
255
256     while (true) {
257         cout << "\nВведите слово для проверки (или 'exit' для выхода): ";
258         cin >> input;
259
260         if (input == "exit" || input == "EXIT") {
261             cout << "Программа завершена." << endl;
262             break;
263         }
264
265         try {
266             if (recognize(input)) {
267                 cout << "\n>>> РЕЗУЛЬТАТ: СЛОВО \"" << input << "\" ПРИНАДЛЕЖИТ ЯЗЫКУ" << endl;
268             }
269             else {
270                 cout << "\n>>> РЕЗУЛЬТАТ: СЛОВО \"" << input << "\" НЕ ПРИНАДЛЕЖИТ ЯЗЫКУ" << endl;
271             }
272         }
273         catch (...) {
274             cout << "\n>>> ОШИБКА при обработке слова" << endl;
275         }
276         cout << "-----" << endl;
277     }
278
279     return 0;
280 }

```

4 Тестирование программы

```
=====
РЕГУЛЯРНОЕ ВЫРАЖЕНИЕ
=====
L = (a+e) (a+b+c+d+e)* (aaa+bbb+ccc+ddd+eee) (a+b+c+d+e)* (a+e)
=====

=====
ФОРМАЛЬНОЕ ОПИСАНИЕ АВТОМАТА
=====
M = (Q, ?, ?, q0, F)

Q = {S, A1, B1, C1, D1, E1, A2, B2, C2, D2, E2, Fa, Fb, Fc, Fd, Fe, R}
? = {a, b, c, d, e}
? - функция переходов (см. таблицу выше)
q0 = S
F = {Fa, Fe} (допускающие состояния)
=====

=====
ОПИСАНИЕ СОСТОЯНИЙ
=====
S - начальное состояние
A1 - последний символ 'a'
B1 - последний символ 'b'
C1 - последний символ 'c'
D1 - последний символ 'd'
E1 - последний символ 'e'
A2 - последние два символа 'aa'
B2 - последние два символа 'bb'
C2 - последние два символа 'cc'
D2 - последние два символа 'dd'
E2 - последние два символа 'ee'
Fa - найдено 3 одинаковых, последний символ 'a' (ДОПУСК)
Fb - найдено 3 одинаковых, последний символ 'b' (НЕ ДОПУСК)
Fc - найдено 3 одинаковых, последний символ 'c' (НЕ ДОПУСК)
Fd - найдено 3 одинаковых, последний символ 'd' (НЕ ДОПУСК)
Fe - найдено 3 одинаковых, последний символ 'e' (ДОПУСК)
R - состояние ошибки (отказ)
=====
```

```
=====
ТАБЛИЦА ПЕРЕХОДОВ (? : Q ? ? ? Q)
=====
```

	a	b	c	d	e
S	A1	R	R	R	E1
A1	A2	B1	C1	D1	E1
B1	A1	B2	C1	D1	E1
C1	A1	B1	C2	D1	E1
D1	A1	B1	C1	D2	E1
E1	A1	B1	C1	D1	E2
A2	Fa	B1	C1	D1	E1
B2	A1	Fb	C1	D1	E1
C2	A1	B1	Fc	D1	E1
D2	A1	B1	C1	Fd	E1
E2	A1	B1	C1	D1	Fe
Fa	Fa	Fb	Fc	Fd	Fe
Fb	Fa	Fb	Fc	Fd	Fe
Fc	Fa	Fb	Fc	Fd	Fe
Fd	Fa	Fb	Fc	Fd	Fe
Fe	Fa	Fb	Fc	Fd	Fe
R	R	R	R	R	

```
=====
ПРОВЕРКА СЛОВ
=====
```

Введите слово для проверки (или 'exit' для выхода): adddddae

--- ПРОЦЕСС РАБОТЫ АВТОМАТА ---

Начальное состояние: S

Читаем 'a': S --a--> A1

Читаем 'd': A1 --d--> D1

Читаем 'd': D1 --d--> D2

Читаем 'd': D2 --d--> Fd

Читаем 'd': Fd --d--> Fd

Читаем 'd': Fd --d--> Fd

Читаем 'a': Fd --a--> Fa

Читаем 'e': Fa --e--> Fe

Конечное состояние: Fe

? Состояние допускающее (есть 3 одинаковых и последний символ - гласная)

>>> РЕЗУЛЬТАТ: СЛОВО "adddddae" ПРИНАДЛЕЖИТ ЯЗЫКУ

Введите слово для проверки (или 'exit' для выхода):

5 Вывод

В ходе лабораторной работы был разработан детерминированный конечный автомат, распознающий язык слов, содержащих три одинаковых символа подряд и начинающихся и заканчивающихся на гласную. Автомат не использует счетчики — вся необходимая информация хранится в состояниях. Программа-анализатор корректно определяет принадлежность слова заданному языку, выводит теоретическую часть (формальное описание, таблицу переходов, граф переходов, регулярное выражение) и пошаговую трассировку работы автомата.

6 Github

<https://github.com/MAKSPOWERO/math>